Chapter ML:III

III. Linear Models

- Logistic Regression
- Loss Computation
- Overfitting
- Regularization

©STEIN 2021

Logistic Regression

Binary Classification Problems

Setting:

- X is a set of feature vectors from an inner product space $\mathbf{X}$, $\mathbf{X} \subseteq \mathbf{R}^p$.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate $c(\mathbf{x})$, which is implicitly given via D, with a logistic model function.

©STEIN 2021

Logistic Regression

Binary Classification Problems

Setting:

- X is a set of feature vectors from an inner product space $\mathbf{X}$, $\mathbf{X} \subseteq \mathbf{R}^p$.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate $c(\mathbf{x})$, which is implicitly given via D, with a logistic model function.

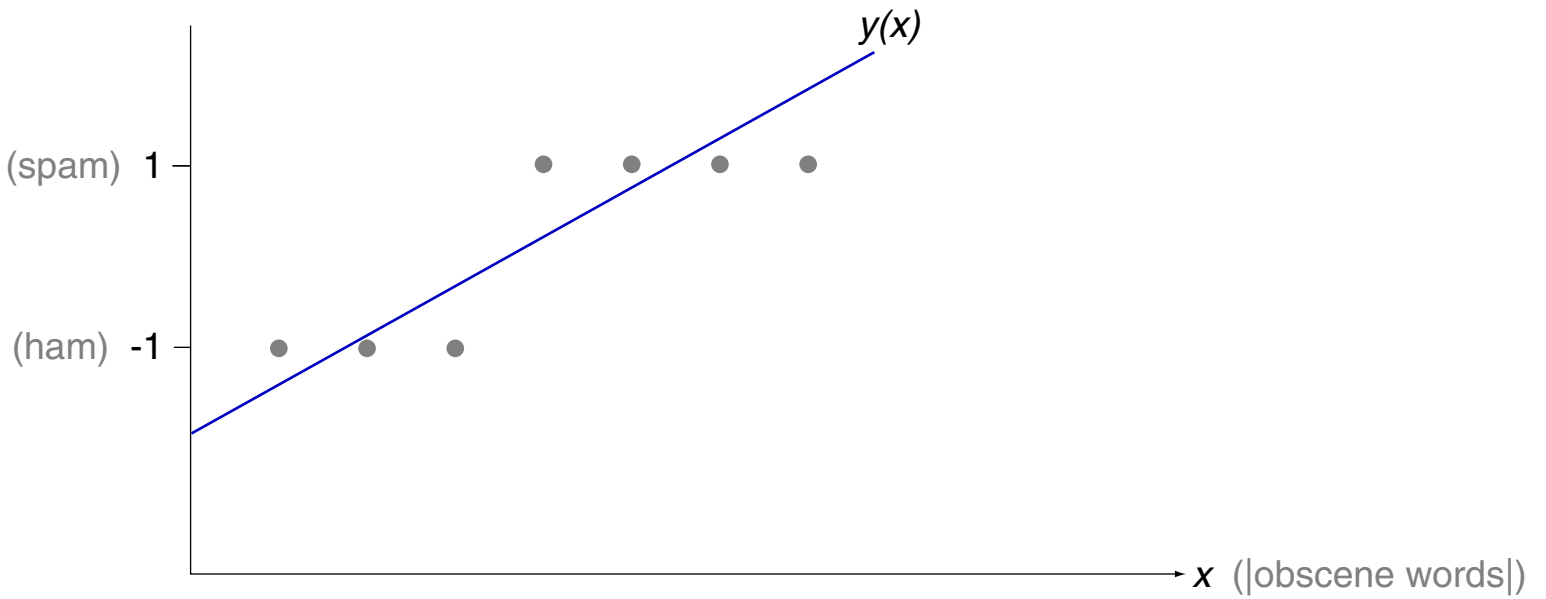
Examples for binary classification problems:

- Is an email spam or ham?
- Is a patient infected or healthy?
- Is a bank customer creditworthy or not?

©STEIN 2021

Logistic Regression

Linear Regression

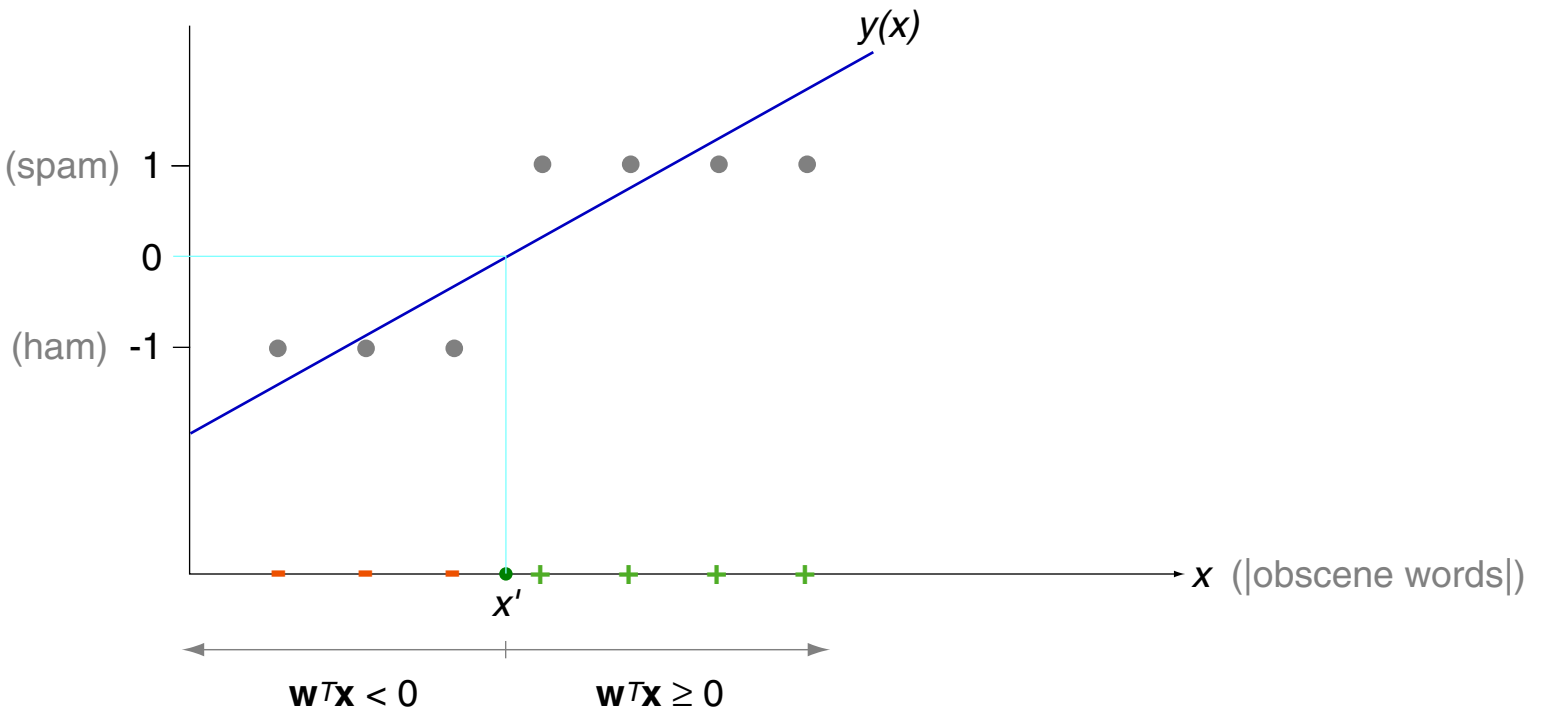


- Linear regression: $y(\mathbf{x}) \overset{(\star)}{=} \mathbf{w}^T\mathbf{x}$

©STEIN 2021

Logistic Regression

Linear Regression (continued)

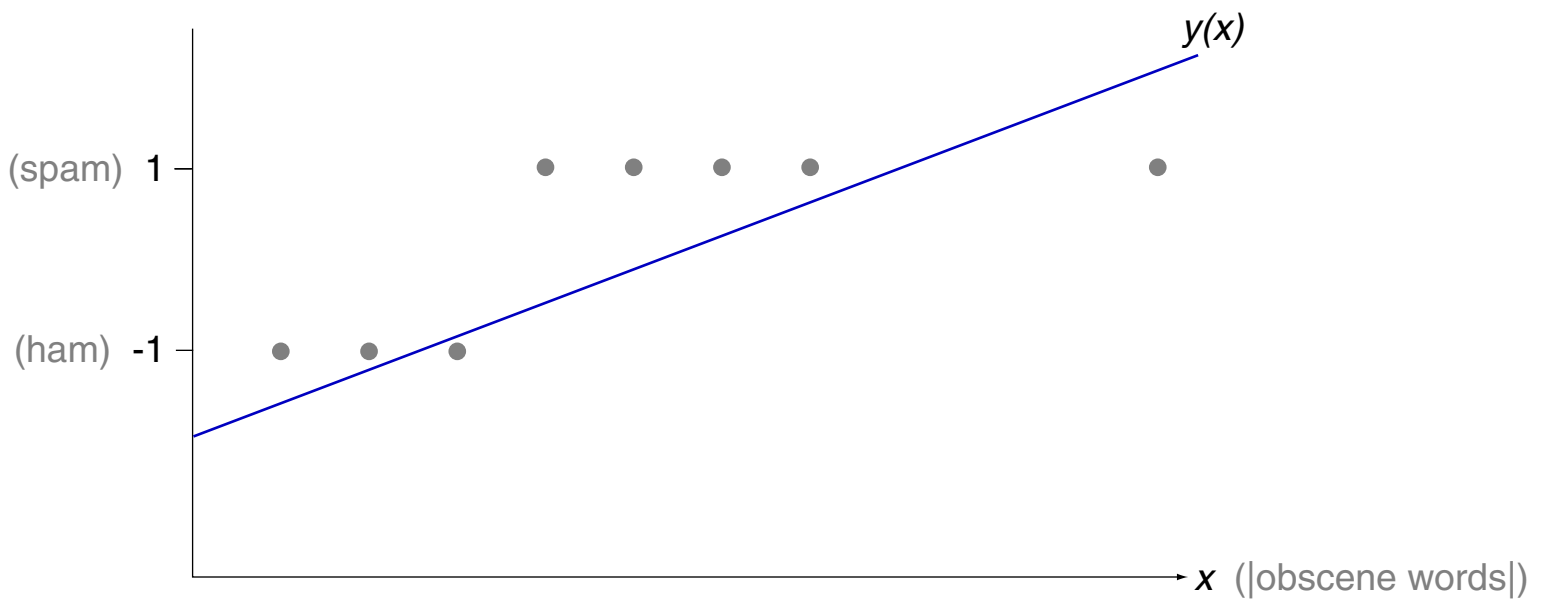


- Linear regression: $y(\mathbf{x}) \overset{(\star)}{=} \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2021

Logistic Regression

Linear Regression (continued)

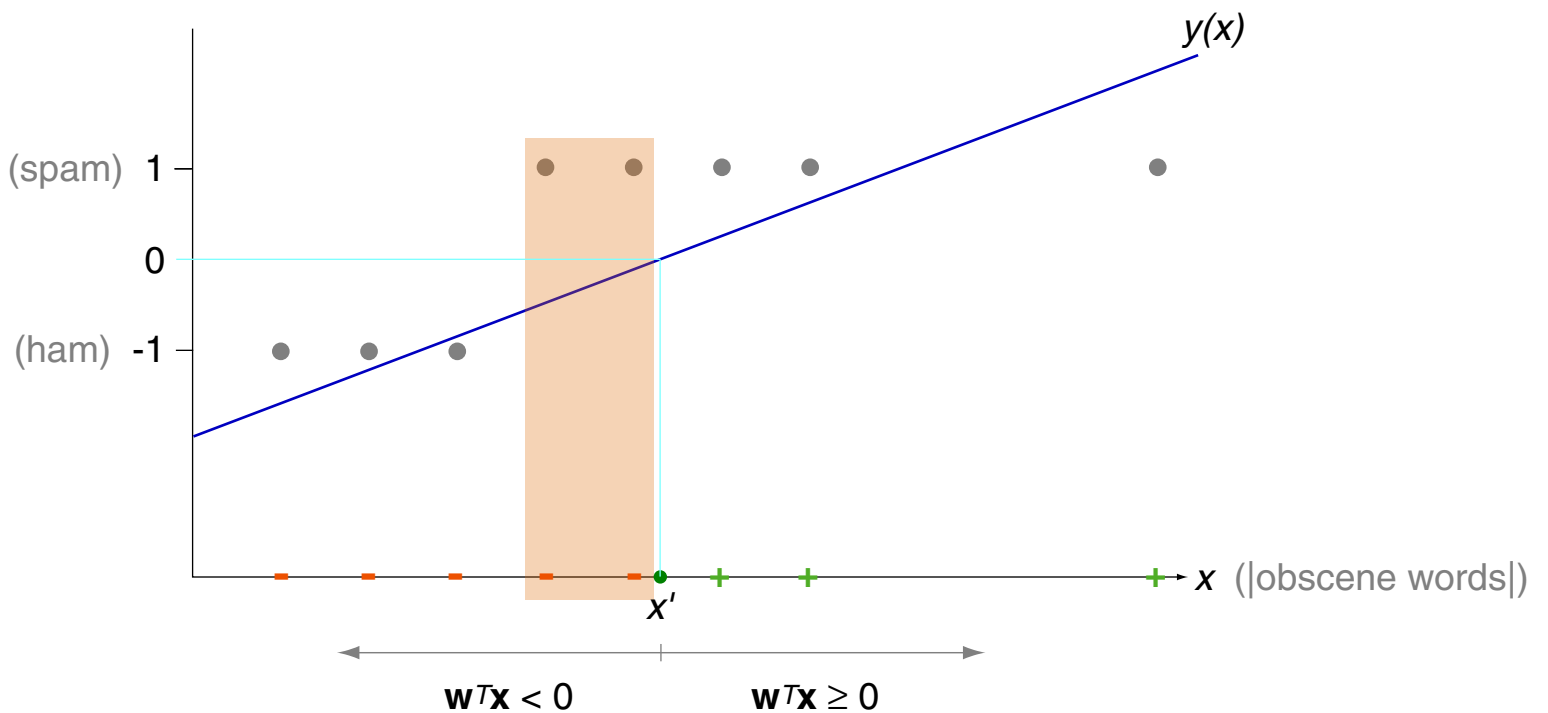


- Linear regression: $y(\mathbf{x}) \overset{(\star)}{=} \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2021

Logistic Regression

Linear Regression (continued)

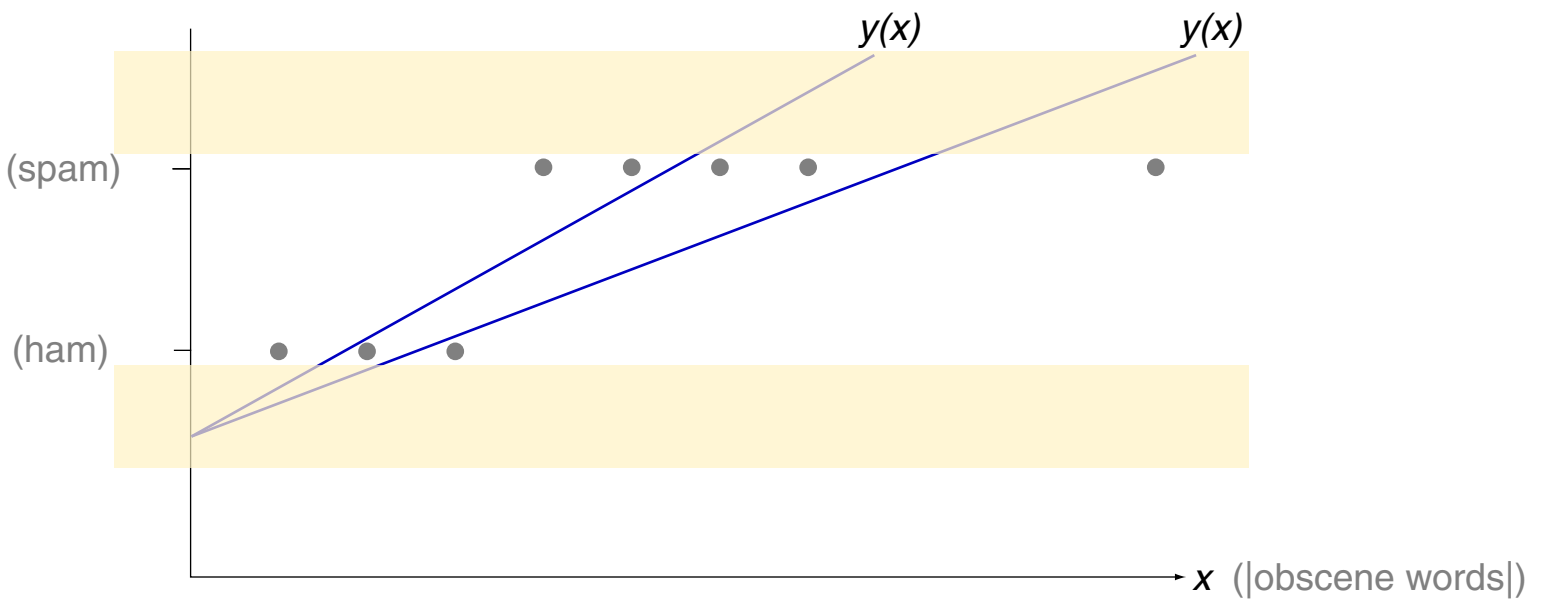


- Linear regression: $y(\mathbf{x}) \overset{(\star)}{=} \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2021

Logistic Regression

Linear Regression (continued)



Restrict the range of $y(\mathbf{x})$ to reflect the two-class classification semantics:

$$-1 \leq y(\mathbf{x}) \leq 1 \qquad \text{or} \qquad 0 \leq y(\mathbf{x}) \leq 1$$

©STEIN 2021

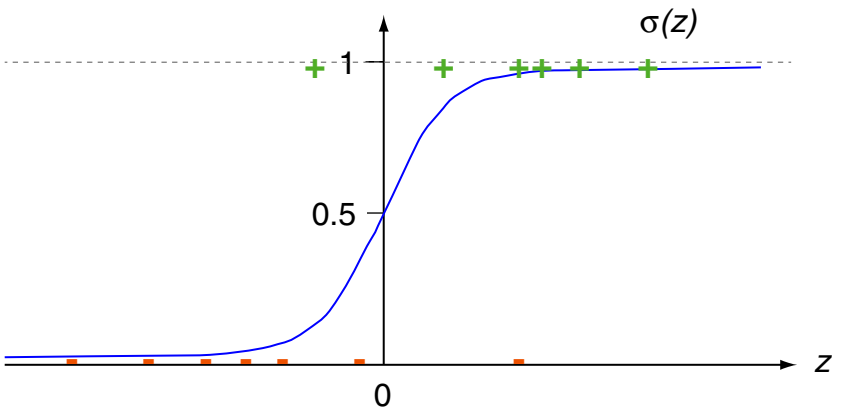
Remarks:

($\star$) Recap. When used as operand in a scalar product with the weight vector, $\mathbf{w}^T\mathbf{x}$, we consider the feature vector $\mathbf{x}$ in its extended form. I.e., $x_0 = 1$, and $\mathbf{x} = (1, x_1, \ldots, x_p)^T \in \mathbf{R}^{p+1}$.

©STEIN 2021

Logistic Regression

Sigmoid (Logistic) Function



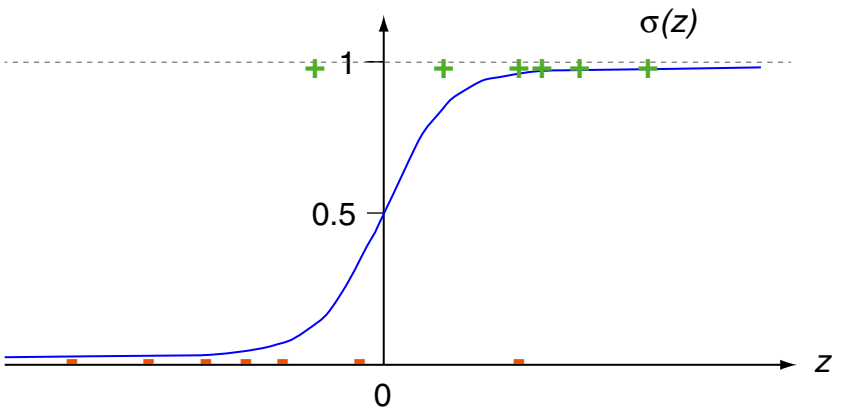
Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

©STEIN 2021

Logistic Regression

Sigmoid (Logistic) Function (continued)



Linear regression

$$\mathbf{w}^T\mathbf{x}$$

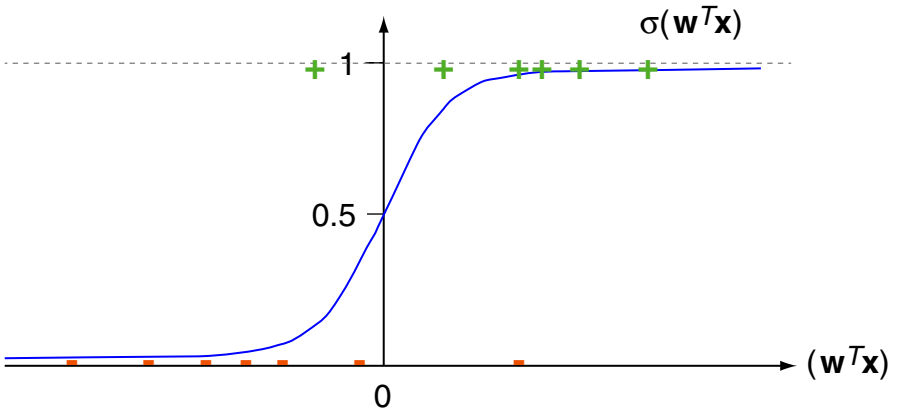
Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

©STEIN 2021

Logistic Regression

Sigmoid (Logistic) Function (continued)



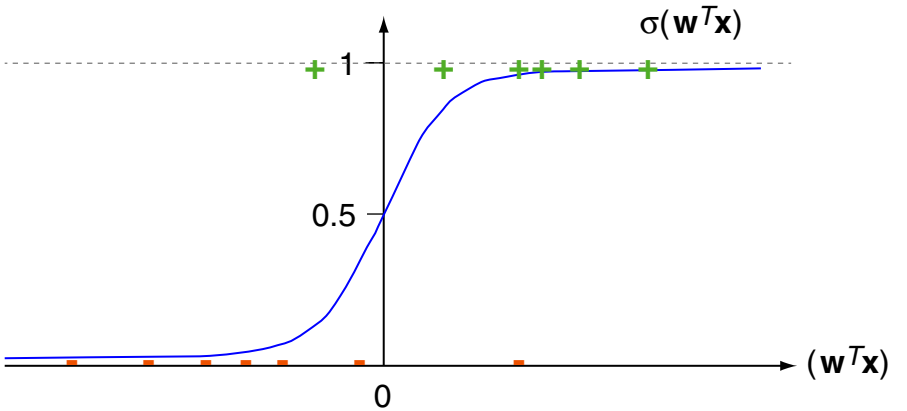
Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) \overset{(\star)}{=} \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

©STEIN 2021

Logistic Regression

Sigmoid (Logistic) Function (continued)



Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) \overset{(\star)}{=} \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

$$y(\mathbf{x}) : \mathbf{R}^{p+1} \rightarrow (0;1)$$

©STEIN 2021

Logistic Regression

Interpretation of the Logistic Model Function

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

©STEIN 2021

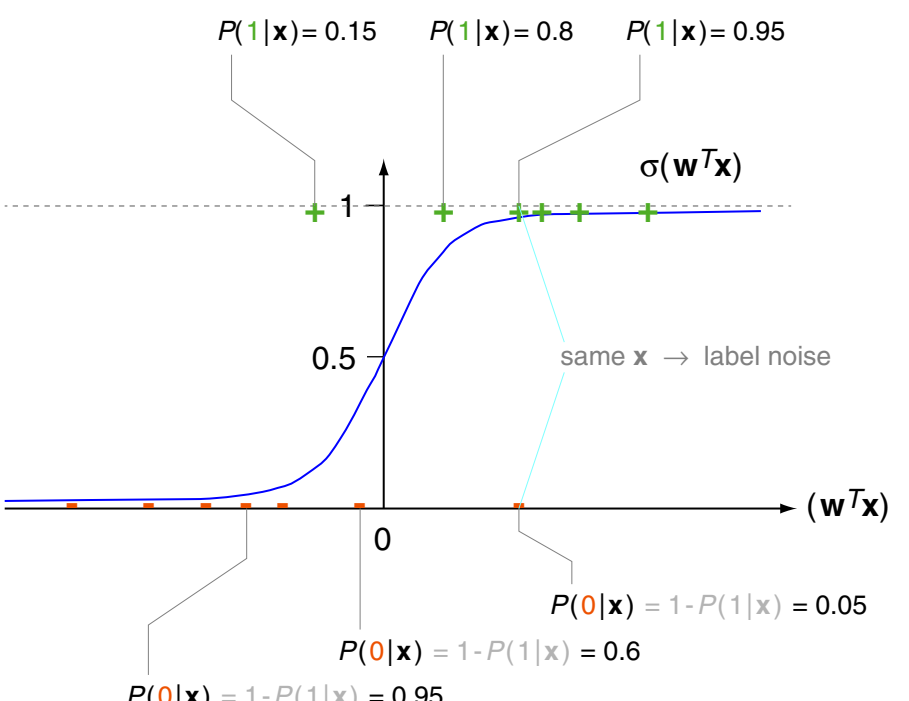
Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Observations D, given as "+" and "-", along with the probability values as specified by $\sigma(\mathbf{w}^T\mathbf{x})$:



©STEIN 2021

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x}){=}0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} \;=\; \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2021

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x})=1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x})=0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} \;=\; \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2021

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x})=1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x})=0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} \;=\; \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2021

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x}) \;=\; \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as the estimated probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$:

- $y(\mathbf{x}) \;=\; P(c(\mathbf{x})=1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."
- $1 - y(\mathbf{x}) \;=\; P(c(\mathbf{x})=0 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 0$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} \;=\; \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \;=\; \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 & \Leftrightarrow \; \mathbf{w}^T\mathbf{x} \geq 0 \\ c(\mathbf{x}) = 0, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 & \Leftrightarrow \; \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2021

Remarks (interpretation of $y(\mathbf{x})$ as probability):

- Note that the interpretation of $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$ as probability for the event $c(\mathbf{x}) = 1$ is not formally grounded but a decision of the modeler. This decision is based on the advantageous properties of the sigmoid function, on practical considerations, and on heuristic simplifications of the real world. Consider for instance the following two aspects:

  1. The sigmoid function implies that with $\mathbf{w}^T\mathbf{x} \to +\infty$ we get $P(c(\mathbf{w}) = 1) \to 1$ or, likewise, with $\mathbf{w}^T\mathbf{x} \to -\infty$ we get $P(y = 1) \to 0$. Though such a strict monotonicity appears self-evident, it need not necessarily correspond to the observed behavior in a particular real world experiment.

  2. The sigmoid function implies a smooth, virtually linear transition from low probability values (around 0.1) to high probability values (around 0.9) as its argument $\mathbf{w}^T\mathbf{x}$ increases.

     This entanglement of the continuous growth of $\mathbf{w}^T\mathbf{x}$ with the continuous growth of probability values $P(c(\mathbf{w}) = 1)$ presumes a proportional connection between cause (in the form of "$\mathbf{x}$") and effect (in the form of "$c(\mathbf{w}) = 1$"). Again, such a relation appears sensible but may not necessarily model the behavior in the real world.

- In section Probability Basics of part Statistical Learning we recapitulate how to formally construct a probability measure given a sample space Ω. Here, in logistic regression, the sample space is the set of all possible observations, i.e., $\Omega = \{(c(\mathbf{x}), \mathbf{x}) \mid c(\mathbf{x}) \in \{0, 1\}, \mathbf{x} \in \mathbf{R}\}$.

©STEIN 2021

Remarks (derivation of $L_\sigma(\mathbf{w})$):

- If $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$ is interpreted as "probability that $c(\mathbf{x})=1$ given feature vector $\mathbf{x}$", then $\mathbf{w}$ is the characterizing parameter vector θ of the (hidden) stochastic process that generates the observed data D.

- Hence, the most probable hypothesis in the space H of possible hypotheses, h_{ML}, can be estimated via the maximum likelihood principle: $h_{\text{ML}} = \text{argmax}_{h\in H}\, P(D \mid h)$.

  Within the logistic regression setting this translates as follows:

$$h_{\text{ML}} \;\equiv\; y_{\text{ML}}(\mathbf{x}) \;\equiv\; \sigma(\mathbf{w}_{\text{ML}}^T\mathbf{x}) \;\leftarrow\; \mathbf{w}_{\text{ML}} = \underset{\mathbf{w}\in\mathbf{R}^{p+1}}{\text{argmax}}\, P(D \mid \mathbf{w}),$$

  where

$$\begin{aligned}
\underset{\mathbf{w}\in\mathbf{R}^{p+1}}{\text{argmax}}\, P(D \mid \mathbf{w}) &= \underset{\mathbf{w}\in\mathbf{R}^{p+1}}{\text{argmax}} \prod_{(\mathbf{x},c(\mathbf{x}))\in D} P(c(\mathbf{x})=c \mid \mathbf{x};\mathbf{w}) \\
&= \underset{\mathbf{w}\in\mathbf{R}^{p+1}}{\text{argmax}} \prod_{(\mathbf{x},1)\in D} \sigma(\mathbf{w}^T\mathbf{x}) \cdot \prod_{(\mathbf{x},0)\in D} (1-\sigma(\mathbf{w}^T\mathbf{x})) \\
&= \underset{\mathbf{w}\in\mathbf{R}^{p+1}}{\text{argmax}} \prod_{(\mathbf{x},1)\in D} y(\mathbf{x}) \cdot \prod_{(\mathbf{x},0)\in D} (1-y(\mathbf{x}))
\end{aligned}$$

©STEIN 2021

Remarks (derivation of $L_\sigma(\mathbf{w})$) (continued):

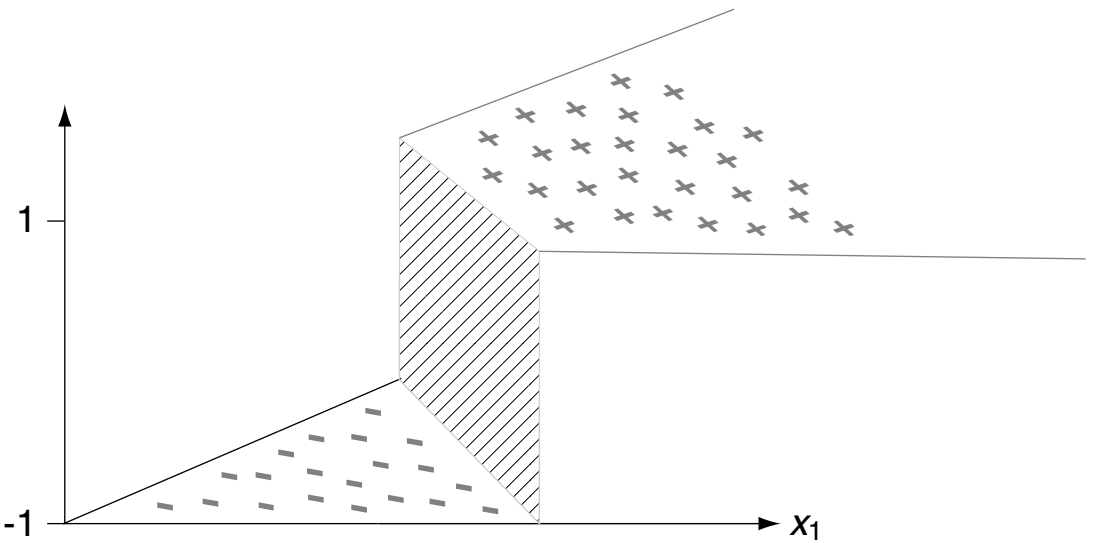
- Note that the maximization problem of the previous page (the argmax-expression) can be reformulated as minimization problem, i.e., as an argmin-expression.
  Obviously, the argument of this argmin-expression quantifies $L(\mathbf{w})$, the global logistic loss of y related to some $\mathbf{w}$, and, similarly, the pointwise logistic loss, $\ell_\sigma(c(\mathbf{x}), y(\mathbf{x}))$. Both of which are derived below:

$$
\begin{aligned}
\operatorname*{argmax}_{\mathbf{w}\in\mathbf{R}^{p+1}} \prod_{(\mathbf{x},1)\in D} y(\mathbf{x}) \cdot \prod_{(\mathbf{x},0)\in D} (1-y(\mathbf{x})) &= \operatorname*{argmax}_{\mathbf{w}\in\mathbf{R}^{p+1}} \log \prod_{(\mathbf{x},1)\in D} y(\mathbf{x}) \;+\; \log \prod_{(\mathbf{x},0)\in D} (1-y(\mathbf{x})) \\
&= \operatorname*{argmax}_{\mathbf{w}} \sum_{(\mathbf{x},1)\in D} \log y(\mathbf{x}) \;+\; \sum_{(\mathbf{x},0)\in D} \log(1-y(\mathbf{x})) \\
&= \operatorname*{argmax}_{\mathbf{w}} \sum_{(\mathbf{x},c(\mathbf{x}))\in D} c(\mathbf{x}) \cdot \log y(\mathbf{x}) \;+\; (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x})) \\
&= \operatorname*{argmin}_{\mathbf{w}} -\sum_{D} c(\mathbf{x}) \cdot \log y(\mathbf{x}) \;+\; (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x})) \\
&= \operatorname*{argmin}_{\mathbf{w}} \sum_{D} \overbrace{-c(\mathbf{x}) \cdot \log(y(\mathbf{x})) \;-\; (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x}))}^{\equiv \ell_\sigma} \\
&\equiv \operatorname*{argmin}_{\mathbf{w}} \sum_{D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) \;=\; \operatorname*{argmin}_{\mathbf{w}} L_\sigma(\mathbf{w})
\end{aligned}
$$

©STEIN 2021

Logistic Regression

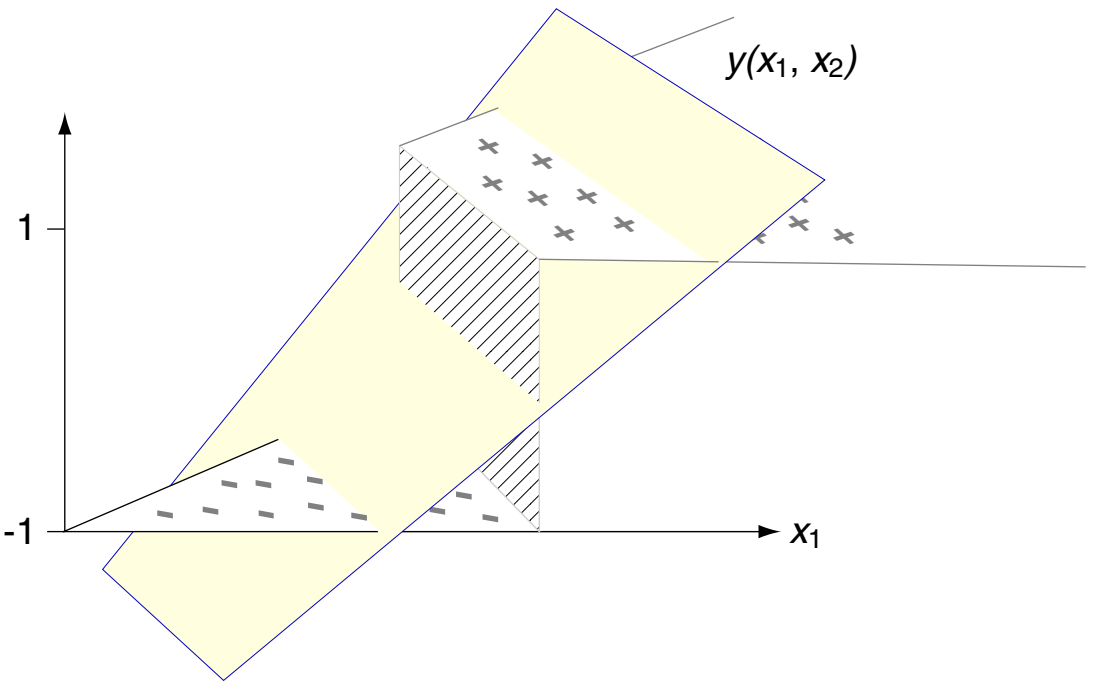
Recap. Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2021

Logistic Regression

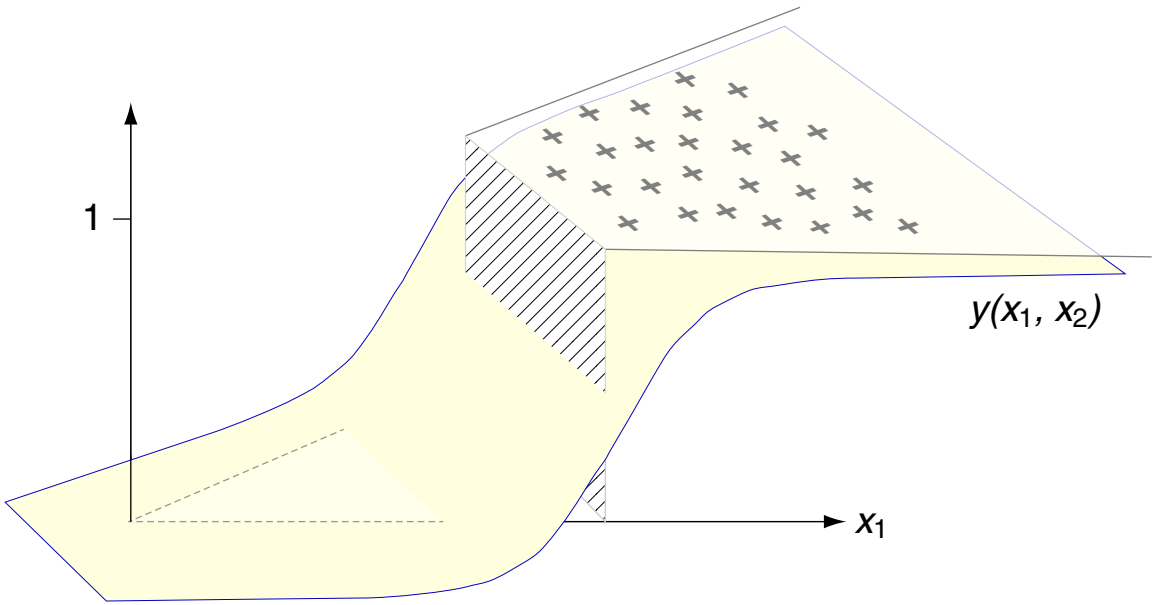
Recap. Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2021

Logistic Regressionn

Logistic Regression for Classification (illustrated for $p = 2$)

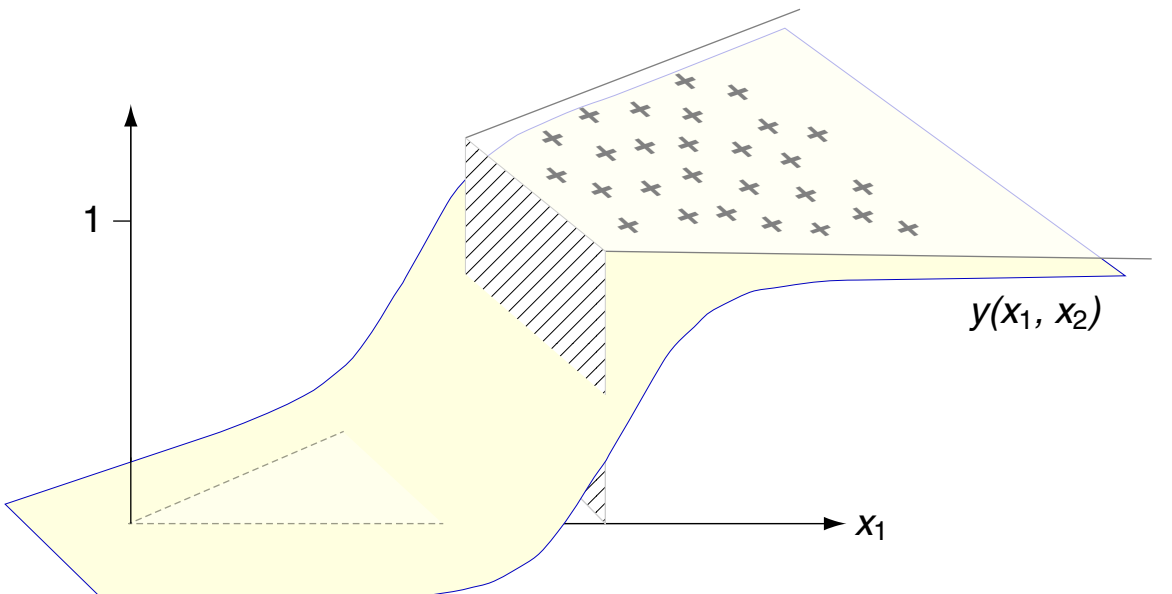


©STEIN 2021

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



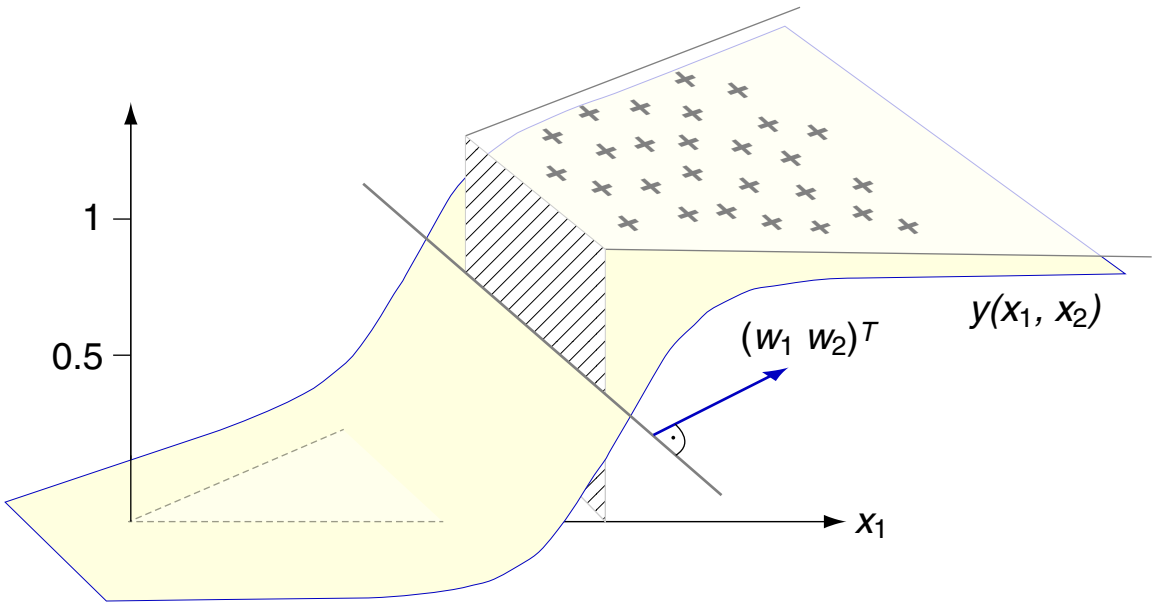
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2021

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



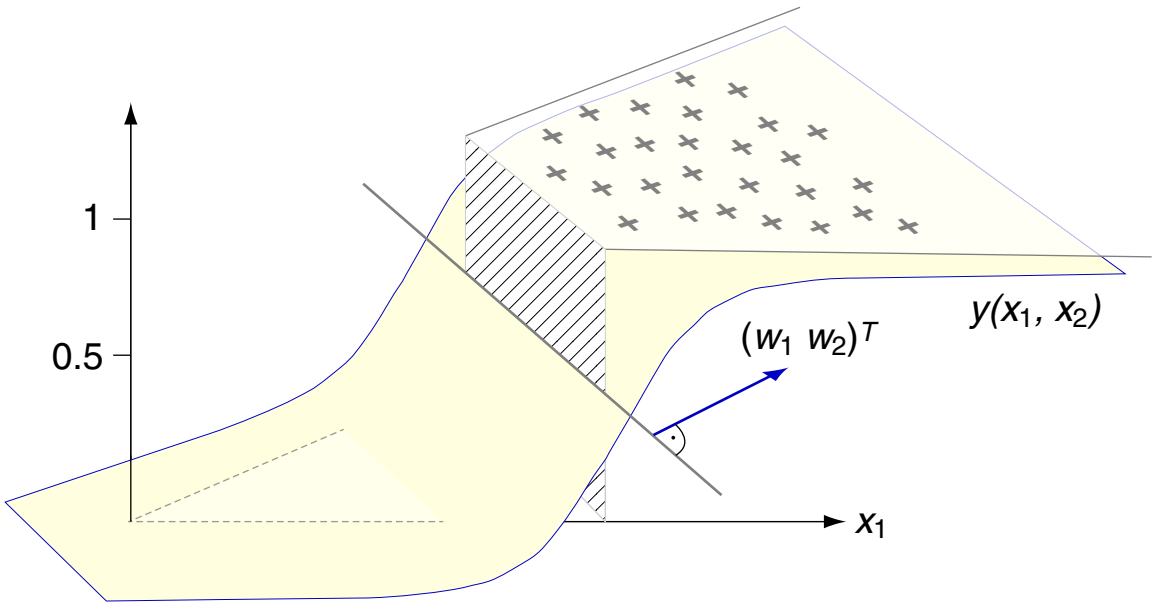
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2021

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



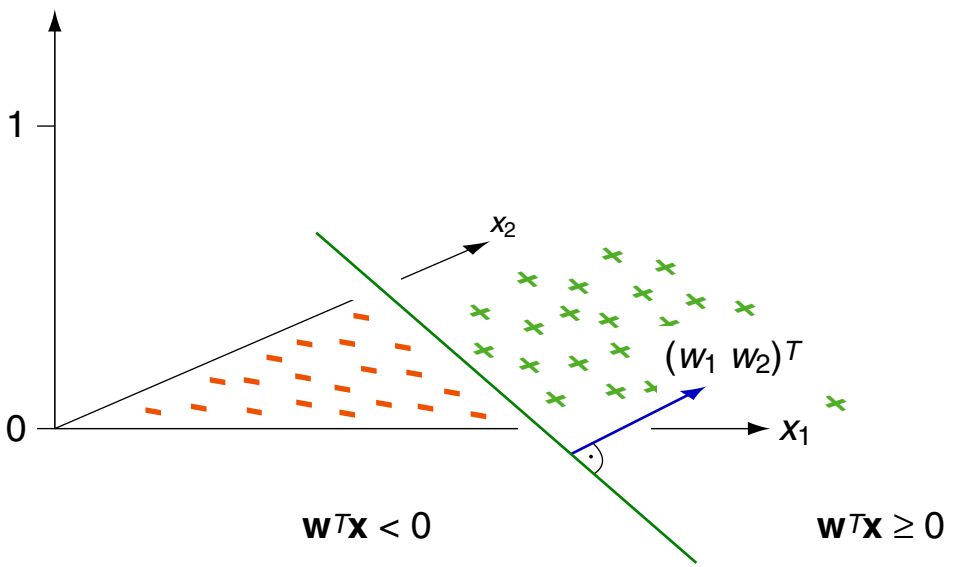
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2021

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



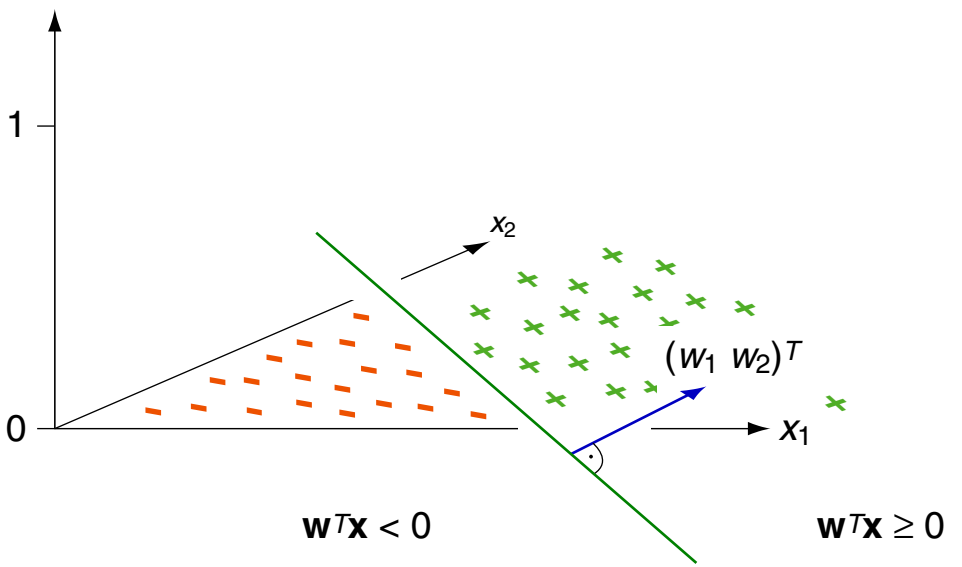
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2021

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.

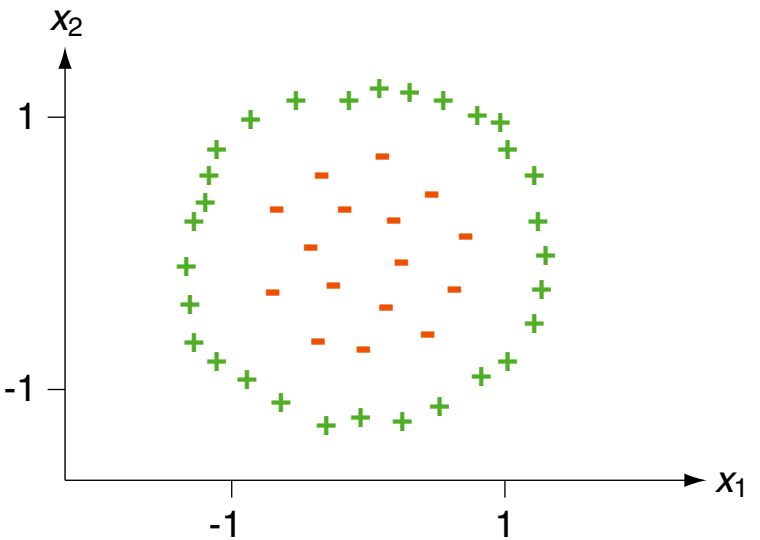


Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} \geq 0 \\ c(\mathbf{x}) = 0, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2021

Logistic Regression

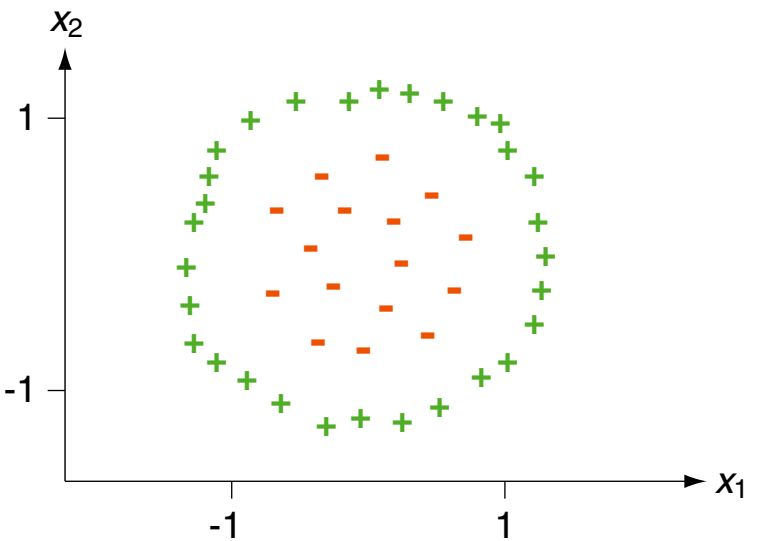
Non-Linear Decision Boundaries



©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



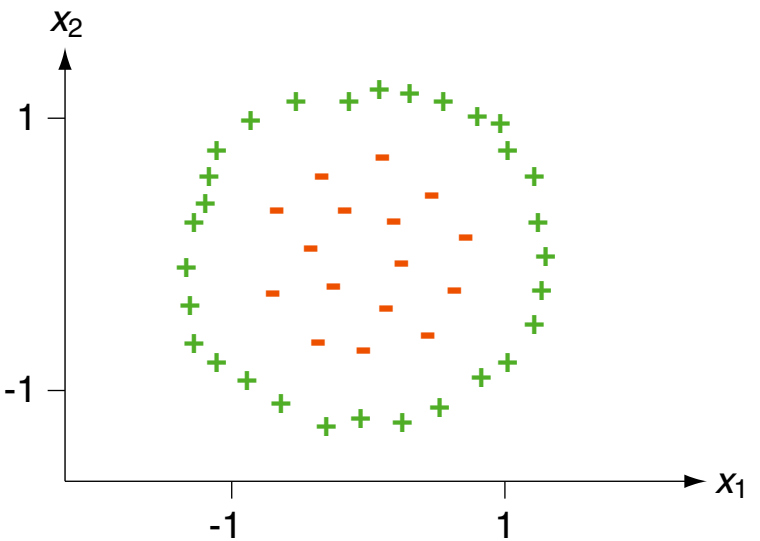
➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



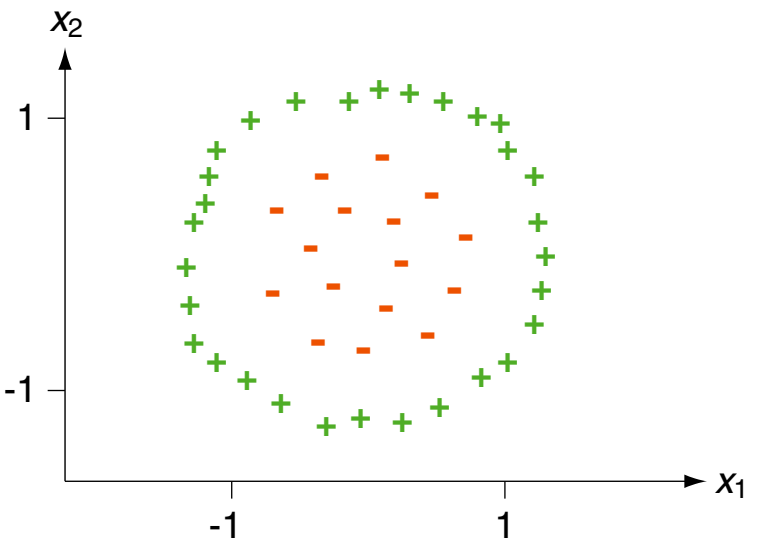
➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

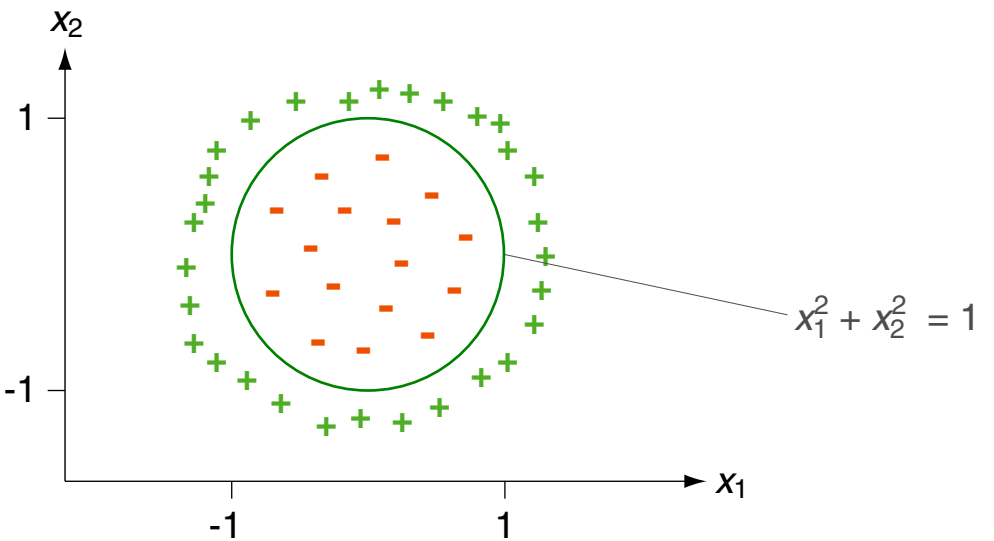
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\quad \rightsquigarrow \quad y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 \,+\, x_1^2 \,+\, x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \;\geq\; 0$

©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

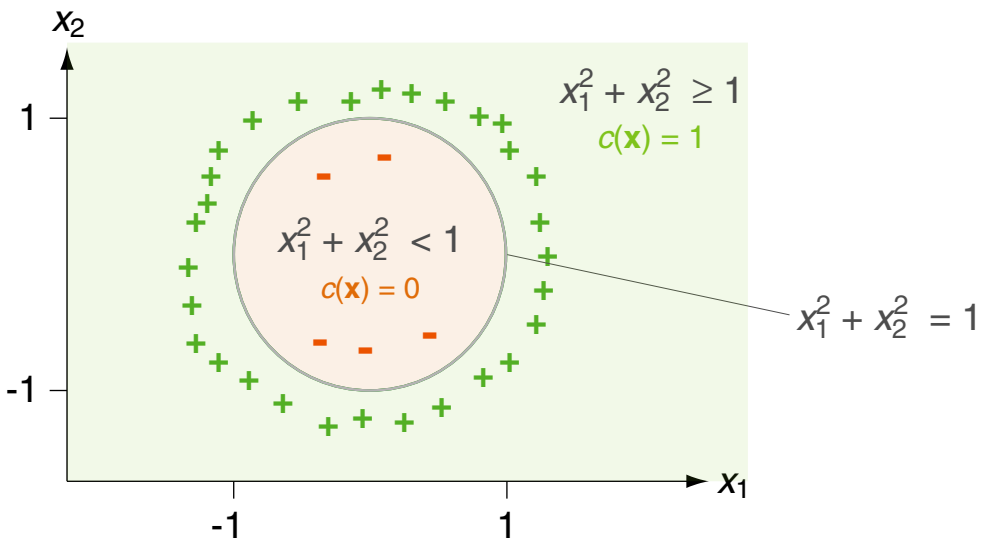
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\rightsquigarrow$ $y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 \,+\, x_1^2 \,+\, x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \;\geq\; 0$

©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

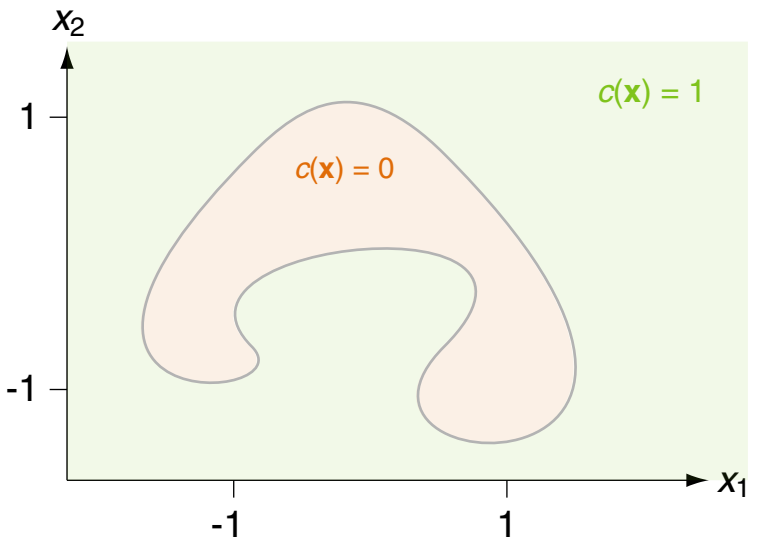
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\quad \rightsquigarrow \quad y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 \,+\, x_1^2 \,+\, x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \;\geq\; 0$

©STEIN 2021

Logistic Regression

Non-Linear Decision Boundaries (continued)



More complex polynomials entail more complex decision boundaries:

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_1^2 \cdot x_2 + w_5 \cdot x_1^2 \cdot x_2^2 + \ldots)$$

©STEIN 2021

Remarks:

- Observe that under logistic regression the structure of a hypothesis, i.e., the forms of possible decision boundaries, is identical to the hypothesis structure under linear regression. Similarly, the respective hypothesis spaces are the same. Hence, the expressiveness, i.e., the complexity of classification problems that can be tackled (or, the effectiveness at which classification problems can be decided) is identical for the two regression approaches.

- Linear regression and logistic regression differ in the way the respective model function parameters, $\mathbf{w}$, are computed. Put another way, the strategy by which the hypothesis space is searched is different. Since both regression methods aim at loss (error, cost) minimization, their different behavior results from their different loss (error, cost) definitions.

- A note on terminology. Instead of “loss” (function) also the terms “error” (function) or “cost” (function) are used, usually with the same semantics as introduced here. We will use the term “error” for classification problems and the term “loss” for both classification and regression problems.

©STEIN 2021

Loss Computation

Loss Computation in Linear Regression

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) = \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) = (c(\mathbf{x}) - y(\mathbf{x}))^2$

©STEIN 2021

Loss Computation

Loss Computation in Linear Regression (continued)

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) \;=\; I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) \;=\; \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) \;=\; (c(\mathbf{x}) - y(\mathbf{x}))^2$

©STEIN 2021

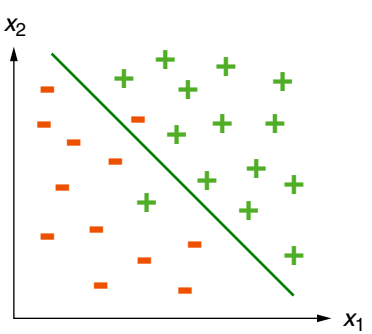
Loss Computation

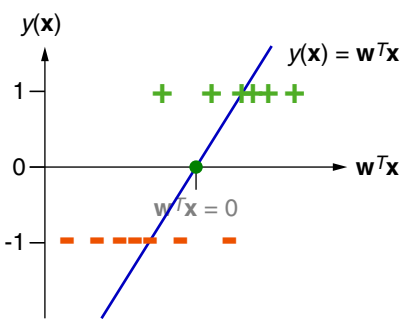
Loss Computation in Linear Regression (continued)

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) = \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) = (c(\mathbf{x}) - y(\mathbf{x}))^2$

Illustration for a particular $\mathbf{w}$:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2021

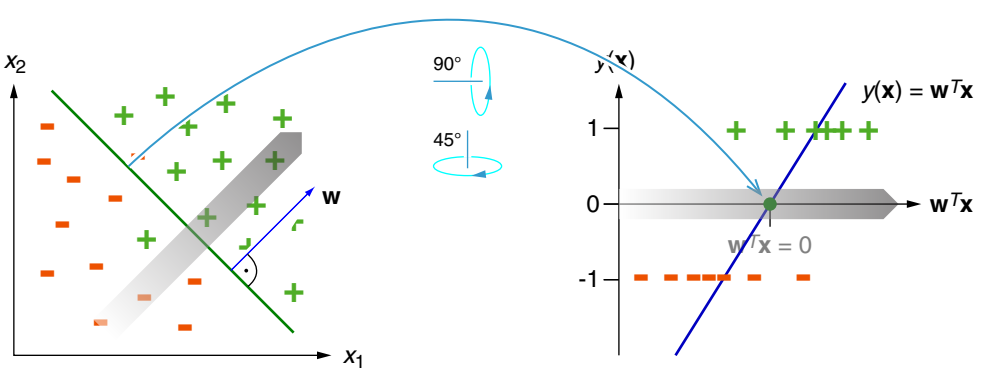
Loss Computation

Loss Computation in Linear Regression (continued)

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) = \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) = (c(\mathbf{x}) - y(\mathbf{x}))^2$

Illustration for a particular $\mathbf{w}$:

Input space: $y(\mathbf{x})$ over hyperplane distance:



©STEIN 2021

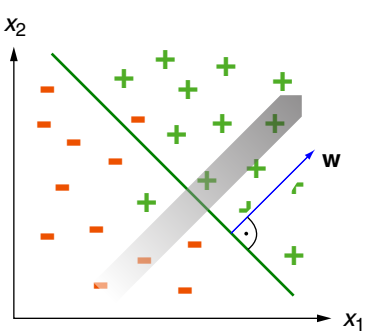
Loss Computation

Loss Computation in Linear Regression (continued)

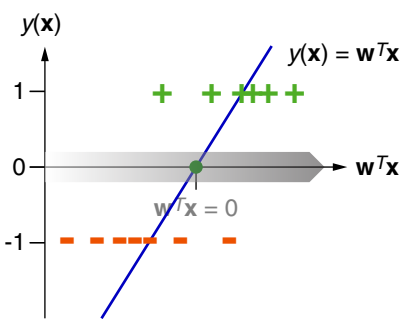
- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) = \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) = (c(\mathbf{x}) - y(\mathbf{x}))^2$

Illustration for a particular $\mathbf{w}$:

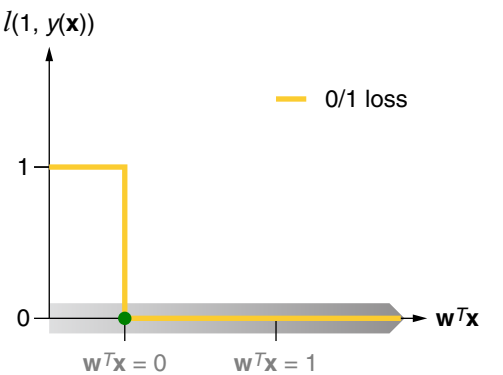
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2021

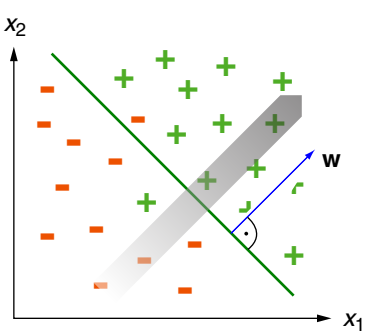
Loss Computation

Loss Computation in Linear Regression (continued)

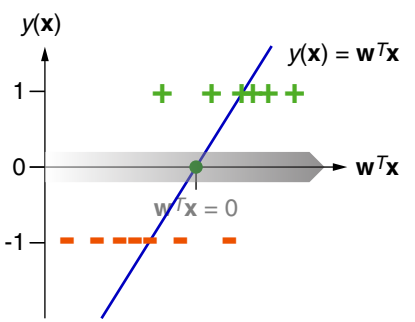
- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) = \begin{cases} 0 & \text{if } c(\mathbf{x}) = \text{sign}(y(\mathbf{x})) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(c(\mathbf{x}), y(\mathbf{x})) = (c(\mathbf{x}) - y(\mathbf{x}))^2$

Illustration for a particular $\mathbf{w}$:

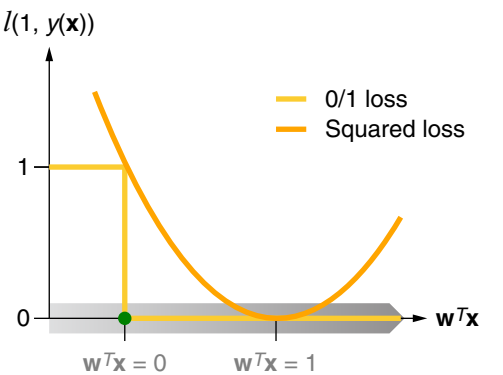
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2021

Remarks:

- We distinguish between (1) the pointwise loss $\ell(c(\mathbf{x}), y(\mathbf{x}))$, which is computed for a single $\mathbf{x}$, and (2) the global loss $L(\mathbf{w})$, which accumulates the pointwise losses of all $\mathbf{x} \in X$ for the weight vector $\mathbf{w}$ used in a specific $y(\mathbf{x})$: $L(\mathbf{w}) = \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \ell(c(\mathbf{x}), y(\mathbf{x}))$.
- Various loss functions for classification and regression problems have been devised.
- The 0/1 loss computes the misclassification error. Recall in this regard the definition of the true misclassification rate.
- The squared loss computes the squared residual. Recall in this regard the residual sum of squares.
- As before, I denotes the indicator function.
- Regarding the illustration: $\mathbf{w}^T\mathbf{x}$ is the hyperplane distance in relation to $||\mathbf{w}||$, the length of $\mathbf{w}$. By scaling $\mathbf{w}$ such that $||\mathbf{w}|| = 1$ the hyperplane distance $\mathbf{w}^T\mathbf{x}$ becomes normalized and is also called “geometric distance”.

©STEIN 2021

Loss Computation

Loss Computation in Logistic Regression

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}) - 0.5))$
  - Logistic loss. $\ell_{\sigma}(c(\mathbf{x}), y(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$ [derivation]

©STEIN 2021

Loss Computation

Loss Computation in Logistic Regression (continued)

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) \;=\; I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}) - 0.5))$
  - Logistic loss. $\ell_{\sigma}(c(\mathbf{x}), y(\mathbf{x})) \;=\; \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$ [derivation]

©STEIN 2021

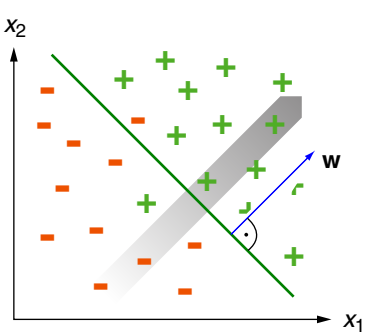
Loss Computation

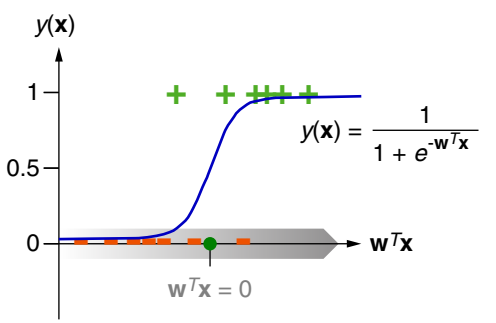
Loss Computation in Logistic Regression (continued)

- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}) - 0.5))$
  - Logistic loss. $\ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$ [derivation]

Illustration for a particular $\mathbf{w}$:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2021

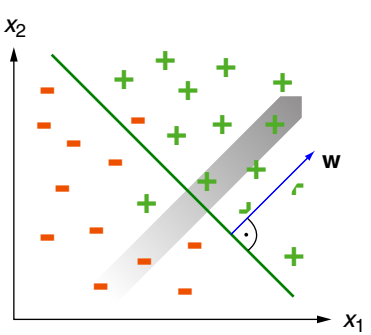
Loss Computation

Loss Computation in Logistic Regression (continued)

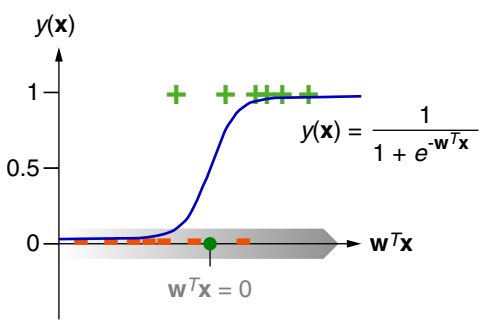
- The pointwise loss, $\ell(c(\mathbf{x}), y(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(c(\mathbf{x}), y(\mathbf{x})) = I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}) - 0.5))$
  - Logistic loss. $\ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$ [derivation]

Illustration for a particular $\mathbf{w}$:

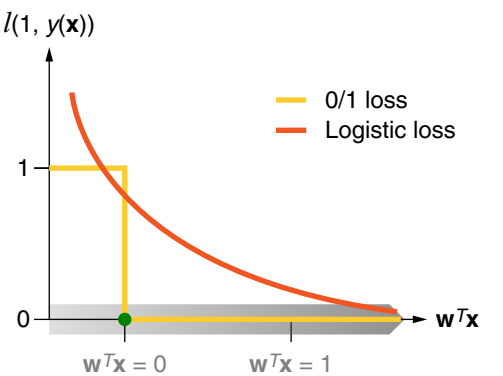
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2021

Remarks:

- The pointwise logistic loss can be rewritten by combining the two cases algebraically:

$$\ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) = -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1 - y(\mathbf{x}))$$

- Observe that loss functions are employed at two places:
  1. For the fitting (the actual regression / optimization / hyperplane search) of the data, where the optimum position of the hyperplane is determined (= a particular hypothesis is selected), and,
  2. for the evaluation of a hypothesis' effectiveness, where the portion of correctly and misclassified examples is analyzed.

- Typically, fitting (optimization) and effectiveness evaluation are done with different loss functions. E.g., logistic regression uses L_σ and $L_{0/1}$ for fitting and evaluation respectively. However, linear regression (not classification) uses RSS (the L_2 loss) for both fitting and evaluation. The basic perceptron learning algorithm uses the misclassification information (the $L_{0/1}$ loss) for both fitting and evaluation.

- By now we have not specified how the parameter vector $\mathbf{w}$ is estimated under the logistic model function. In particular, there is no direct method (recall the normal equations in linear regression) to solve the model function $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$ to obtain the optimum $\hat{\mathbf{w}}$.

©STEIN 2021

Loss Computation

The BGD Algorithm

[Algorithms: LMS BGD$_\sigma$ PT BGD$_2$ IGD]

Algorithm:	BGD$_\sigma$	Batch Gradient Descent.
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\mathbf{x} \in \mathbf{R}^p$, $c(\mathbf{x}) \in \{0, 1\}$.
	η	Learning rate, a small positive constant.
Output:	$\mathbf{w}$	Weight vector from $\mathbf{R}^{p+1}$ (= hypothesis).

BGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
   t = t + 1
   Δw = 0
   FOREACH (x, c(x)) ∈ D DO
      δ = c(x) − y(x)      // y(x) ⁽ˣ⁾= σ(wᵀx) = 1/(1+e^(−wᵀx)).
      Δw ⁽ˣ⁾= Δw + η · δ · x    // δ · x is the derivative of ℓ_σ(c(x), y(x)).
   ENDDO
   w = w + Δw
UNTIL(convergence(c(D), y(D)) OR t > t_max)
return(w)
```


©STEIN 2021

Remarks:

- Each BGD iteration "`REPEAT ... UNTIL`"
  1. computes the direction of steepest loss descent as $-\nabla L_\sigma(\mathbf{w}_t) = \sum_{(\mathbf{x},c(\mathbf{x}))\in D} (c(\mathbf{x}) - y_t(\mathbf{x})) \cdot \mathbf{x}$, and
  2. updates $\mathbf{w}_t$ by taking a step of length η in this direction.

- Line 10: The *convergence* function may analyze the global logistic loss, $L_\sigma(\mathbf{w}_t)$, or the norm of the loss gradient, $||\nabla L_\sigma(\mathbf{w}_t)||$, and compare it to some small positive bound ε. In this regard $c(D)$ and $y(D)$ denote the vectors of target classes and computed values respectively.

- The mathematical derivation of $(c(\mathbf{x}) - y(\mathbf{x})) \cdot \mathbf{x}$, which literally is the derivative of the pointwise logistic loss $\ell_\sigma(c(\mathbf{x}), y(\mathbf{x}))$, as well as notes regarding the speed of convergence of the basic gradient descent are given in section Gradient Descent of part Neural Networks.

©STEIN 2021

Loss Computation

Logistic Regression in a Nutshell

Optimization approach

Optimization objective
Loss function [+ Regularization]

Model function $\rightsquigarrow$ Hypothesis space

Task

Data

©STEIN 2021

Loss Computation

Logistic Regression in a Nutshell (continued)

Optimization approach

Optimization objective
Loss function [+ Regularization]

Model function $\rightsquigarrow$ Hypothesis space

Task

Binary classification

Data

$$D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times \{0, 1\}$$

©STEIN 2021

Loss Computation

Logistic Regression in a Nutshell (continued)

Optimization approach

Optimization objective
Loss function [+ Regularization]

Model function $\rightsquigarrow$ Hypothesis space

- Hypothesis space: Set of linear maps
- Logistic model: $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$

Task

Binary classification

Data

$D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times \{0, 1\}$

©STEIN 2021

Loss Computation

Logistic Regression in a Nutshell (continued)

Optimization approach

Optimization objective

Loss function [+ Regularization]

- Objective: Minimize logistic loss
- Regularization: None
- Loss: $-c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x}))$

Model function $\rightsquigarrow$ Hypothesis space

- Hypothesis space: Set of linear maps
- Logistic model: $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$

Task

Binary classification

Data

$D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times \{0, 1\}$

©STEIN 2021

Loss Computation

Logistic Regression in a Nutshell (continued)

Optimization approach	BGD, Newton-Raphson, BFGS, Conjugate GD
Optimization objective Loss function [+ Regularization]	❑ Objective: Minimize logistic loss ❑ Regularization: None ❑ Loss: $-c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x}))$
Model function $\rightsquigarrow$ Hypothesis space	❑ Hypothesis space: Set of linear maps ❑ Logistic model: $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$
Task	Binary classification
Data	$D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times \{0,1\}$

©STEIN 2021